

AI5707 Assignment 3

Dev Patel
26120007

1. Introduction

The task is to recover a polynomial well enough from the batch of noisy data points that a curve captures the underlying shape without chasing the noise on top of it. The purpose of the exercise is to implement polynomial regression, and to study how the degree of the polynomial affects the quality of the fit, which is where the train/test/validation split and the underfitting-vs-overfitting question comes in.

2. Experiments performed

2.1 Splitting the data

The data was split into 60% train, 20% test and 20% validation (6000 / 2000 / 2000 respectively), shuffled with a fixed random seed so the split is reproducible.

2.2 Fitting polynomials of increasing degree

The regression method is ordinary least squares through the normal equations: build the design matrix $[1, x, x^2, \dots, x^d]$ for each row, form $X^T X$ and $X^T y$ with nested-loop matrix multiplication, and solve $(X^T X)w = X^T y$ using Gauss-Jordan elimination with partial pivoting. Every row is cleared per pivot column, not just the rows below it, so the solution can be read directly off the last column with no separate back-substitution step. Regression is fit directly on the raw x and y values, with no normalisation; testing across a wide range of degrees confirmed the solver stays stable without it, up to a floating-point limit described below.

Degrees 1 through 75 were fitted in a single sweep. The table below shows the first 20 for readability; the full log is in results.txt.

Degree	Train MSE	Test MSE
1	2121.24	2144.70
2	2094.03	2119.60
3	1964.20	1978.47
4	1826.87	1880.26
5	1624.90	1653.92
6	1615.49	1642.16
7	1317.38	1357.78
8	1124.99	1138.01
9	830.34	853.21
10	830.34	853.30
11	830.23	853.18
12	830.20	853.07
13	830.06	852.75
14	830.05	852.79
15	830.05	852.78
16	829.90	852.64
17	829.30	855.16
18	829.30	855.12
19	829.11	854.96
20	829.11	854.85

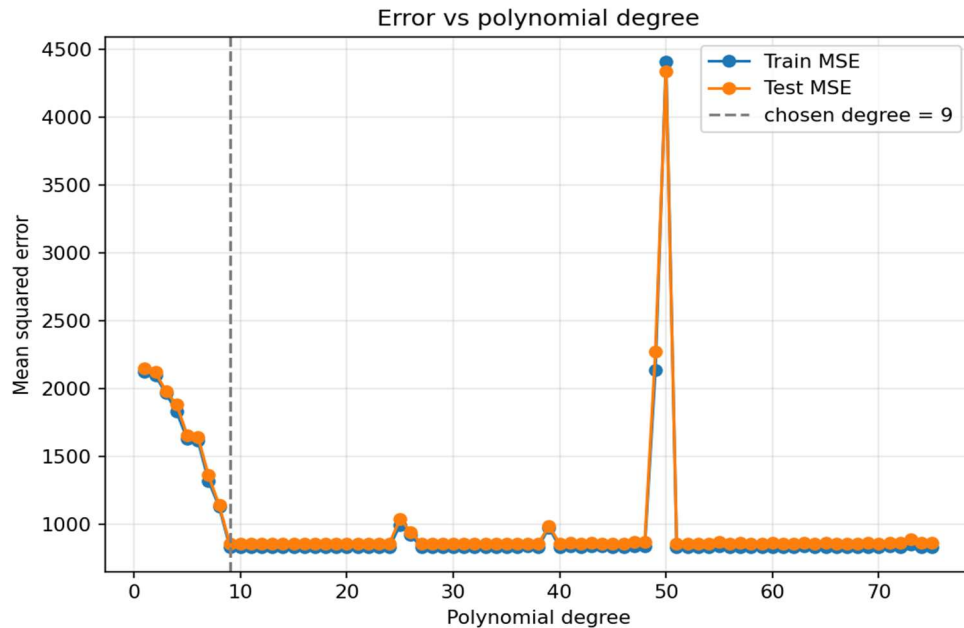


Figure 1: Train and test MSE as the polynomial degree increases.

Figure 1 shows both errors falling sharply up to about degree 8, then flattening into a long plateau from degree 9 onward. The lowest test MSE, strictly speaking, is at degree 16, but the numbers from degree 9 through 16 are 853.21 down to 852.64, a spread of about 0.07%, well inside the noise range one can expect from a 2000-point test set. So I picked the degree where the test MSE curve stops improving by more than 1% per step, which is degree 9, visibly the point where the plateau begins. A few sharp spikes appear later on the plateau, most visibly around degree 50. These presumably come from the solver becoming less well-conditioned at those degrees on unnormalised data, not from overfitting, since train and test error rise together there rather than diverging from each other.

2.3 Final fit and validation performance

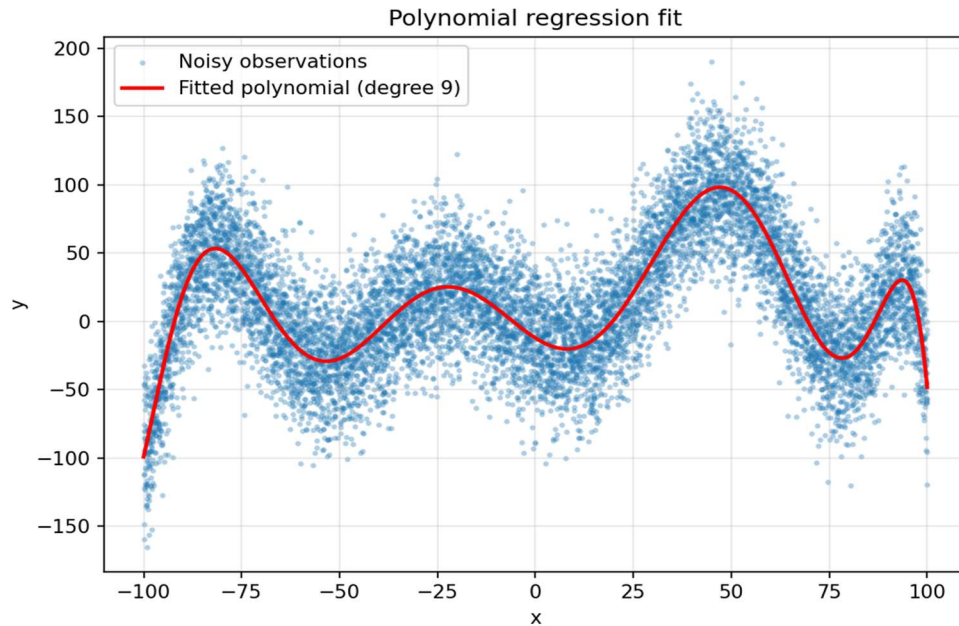


Figure 2: Degree-9 polynomial fitted on the training set, plotted over all 10,000 points.

Figure 2 overlays the degree-9 fit (trained only on the 6000 training points) on the full dataset. The red curve tracks the bumps and dips in the scatter closely. It visibly bends to follow the oscillations that are present in the data, without hugging every individual noisy point. Numerically, this model reaches:

- Train MSE: 830.34
- Test MSE: 853.21
- Validation MSE: 863.48

The three numbers being close together (830 / 853 / 863) is the main thing to read from this: it means the model generalises rather than memorising the training points.

2.4 Visualising underfitting and overfitting

I also fitted degree 2 and degree 16 and plotted all three side by side against the same data, to see the underfitting/overfitting story visually rather than just as a table of numbers. Panel labels are computed from each degree's actual train and test MSE relative to the chosen fit, not assumed from the degree number as overfitting specifically means test error rising while train error falls, and none of the 75 degrees tested ever show that.

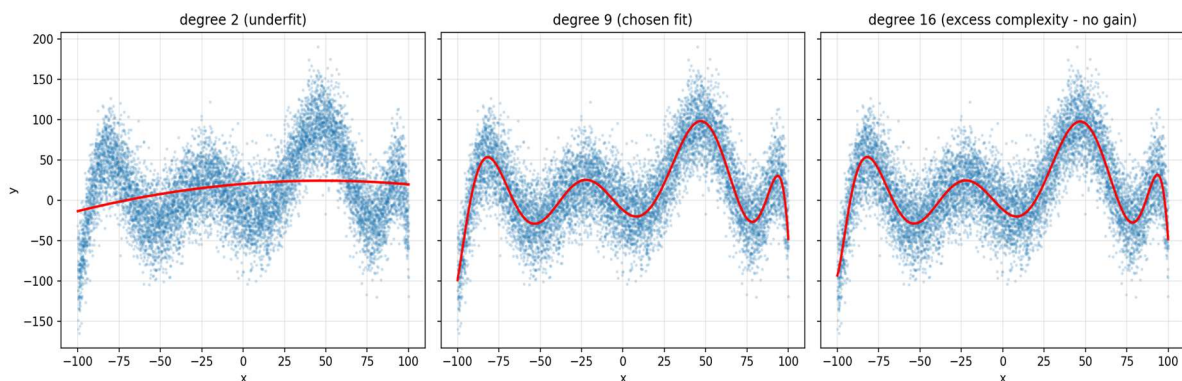


Figure 3: Degree 2 (too simple), degree 9 (chosen), degree 16 (excess complexity - no gain).

Figure 3 makes the degree-2 case obvious. It produces a smooth arc and completely misses the oscillations that are visibly present in the scatter, a classic case of underfit. Degree 9 tracks the bumps well. Degree 16 doesn't look dramatically different in the plot. With 6000 training points the model doesn't have much room to contort itself around individual noisy samples, so the curve looks almost identical to degree 9.

The sweep stops at degree 75. Testing further separately showed the solver starts producing NaN at degree 77: $X^T X$'s largest entry crosses float64's representable range right around there, and the elimination step then produces NaN from operations like infinity minus infinity. That is a floating-point limit, not overfitting, and 75 was chosen as a round number safely below it.

3. Inference

The main learning from this assignment was less about the mechanics of least squares, which is fairly mechanical once the design matrix and normal equations are set up, and more about degree selection being a judgement call, not a pure optimisation. Simply taking $\text{argmin}(\text{test MSE})$ would have handed me degree 16 with seven extra coefficients doing almost nothing, for a 0.07% improvement over degree 9 that's indistinguishable from test-set noise. Preferring the plateau point over the strict minimum is a small change in logic that makes a real difference to how defensible the final model is.

A second, related learning was that numerical instability is something to verify, not assume. It's easy to read '9th and 10th powers of x up to 100' and assume that has to be a problem, but the only way to actually know is to try it and check, in this case across a wide range of degrees, not just the one being reported. The solver turned out to be stable all the way to degree 76, with a specific, defensible limit at degree 77 where float64 itself runs out of range.

Third, the underfit/overfit comparison in Figure 3 was a reminder that visual inspection alone isn't reliable for detecting overfitting once you have enough training data. Degree 9 and degree 16 look almost the same in the plot, and only the train/test gap in the numbers would catch the extra complexity if it mattered. The risk of overfitting is not just a property of the model, it's a property of how much data you have relative to the model's flexibility.